ModelForge: A No-Code Platform for Transparent Machine Learning Pipeline Building with Synchronized Code Generation

[Vibhuti Agarwal]
[CSE(AI)]
[KIET GROUP OF INSTITUTIONS]

[Shubhi Gupta]
[CSE(AI)]
[KIET GROUP OF INSTITUTIONS]

[Sonali Tyagi]
[CSE(AI)]
[KIET GROUP OF INSTITUTIONS]

[Aadya Garg]
[CSE(AI)]
[KIET GROUP OF INSTITUTIONS]

***Abstract*— No-code machine learning (ML) tools lower the barrier to building models, but many return a finished model without showing how it was built, leaving non-expert users unable to debug, trust, or learn from the result. This paper presents ModelForge, a visual, block-based pipeline builder in which a JSON pipeline specification is the single source of truth: the same specification drives training, live metrics, and a synchronized, human-readable code view. A rules-based dataset profiler suggests preprocessing steps that the user must explicitly accept or edit. A large language model (LLM) is restricted to two bounded tasks: drafting an initial pipeline with pre-filled, validated parameters, and drafting reviewable code for preprocessing needs that no pre-built block covers. We describe the system design and propose a controlled study with novice learners that measures conceptual understanding, trust, usability, and mental workload under transparent and non-transparent conditions.**

I. INTRODUCTION

Building an ML model still demands skills that many founders, students, and non-technical builders lack. No-code tools address part of the problem, but many behave as black boxes: the user receives a trained model without seeing the preprocessing, model choice, or code behind it. Such a model is hard to debug, trust, or learn from.

Transparency matters. In a study of data scientists, adding transparency features to an AutoML system increased trust and understandability, and performance metrics and visualizations were rated as the most important information for establishing trust [1]. That study, however, involved data scientists and measured willingness to deploy a model rather than learning.

Educational and no-code ML tools have been studied from several angles, but the available evidence consists mostly of proof-of-concept systems, exploratory classroom reports, and usability or trust measures (Section II). Controlled evidence on whether transparent tooling improves the measured conceptual learning of complete novices appears to be limited.

This paper makes three contributions:

1) **System.** ModelForge, a no-code platform whose pipeline specification is the single source of truth for training, metrics, and generated code.
2) **Bounded use of AI.** A design that separates deterministic, rules-based suggestions from LLM-assisted steps, each shown to the user and never silently applied.
3) **Evaluation protocol.** A controlled study design for measuring novices' conceptual learning, trust, usability, and mental workload.

II. RELATED WORK

Visual programming for ML. Visual Blocks for ML (formerly Rapsai) is a no-code node-graph editor for prototyping ML-based multimedia applications, evaluated through case studies with 15 ML practitioners [2]. It validates the diagram-based approach, but it targets multimedia prototyping by practitioners rather than the teaching of tabular ML to beginners.

LLM-assisted pipeline generation. InstructPipe generates Visual Blocks pipelines from text instructions: LLM modules produce pseudocode that an interpreter renders in the node-graph editor [3]. Its authors report that it did not significantly reduce novice users' mental demand, attributing this to the difficulty of writing good prompts, the burden of switching between verbal and visual modes, and the effort of debugging generated pipelines. It generates graph structure but not node parameters. A controlled, human-in-the-loop agentic framework similarly guides non-AI-expert scientists in building pipelines and their code [4], and a layered architecture for scientific workflows confines LLM non-determinism to intent extraction while validated generators produce the workflow itself [5]. ModelForge follows this separation and does not make prompting its primary interaction.

Low-code and no-code ML. LowCoder combines a visual block interface for scikit-learn pipelines with a natural-language operator search, gives immediate feedback on the effect of each change, and was evaluated with 20 participants of varying expertise [6]. Its authors found that AI helped users who knew what they wanted but did not support novices who lacked clarity about their goal, and they suggest dataset-aware contextual suggestions as future work. Truss and Schmitt use design science and a case study to show that no-code AutoML can support non-experts in AI prototyping

[7]. Teachable Machine lets people explore classification without coding [8], with its educational value supported mainly by uptake rather than a controlled learning measure.

Education and explainability. A classroom report compared KNIME with a GenAI chatbot for teaching ML workflows; surveys, interviews, and observations suggested that both supported conceptual learning, but the study was exploratory and students chose their own tool [9]. The PHOTON Wizard translates GUI interactions into valid source code for the PHOTON framework, as a proof-of-concept aimed at users who already have theoretical ML knowledge [10]. DashAI added an explainability module to a no-code ML platform and evaluated it with 20 novices and experts, measuring usability, explanation satisfaction, and trust [11].

Positioning. ModelForge differs from this work by (i) targeting complete novices, (ii) combining synchronized code, per-block explanations, and dataset-driven suggestions with accept/skip control, (iii) restricting the LLM to bounded, reviewable roles, and (iv) evaluating conceptual learning in a controlled comparison.

III. SYSTEM DESIGN

A. Overview

ModelForge has three layers. The *frontend* (React with a flow-diagram library) lets users drag, connect, and configure blocks (Load Dataset, Preprocessing, Model, Output). The *backend* (Python, FastAPI) stores the pipeline specification, trains it, streams metrics, and generates code. The *AI layer* profiles the dataset, proposes preprocessing steps, and handles the bounded LLM tasks. The frontend exchanges the pipeline specification and results with the backend, and the backend exchanges dataset profiles and suggestions with the AI layer.

B. Pipeline Specification as Single Source of Truth

Every pipeline is a JSON object: an ordered list of blocks, each with a type, an identifier, and parameters. Training, metrics, and code are all derived from this object and are never edited independently. Code is generated from the specification and is never parsed back into blocks, so the diagram and the code cannot drift apart.

C. Code Generation

Most scikit-learn blocks share the form `ClassName(param=value)`. A lookup table maps each block type to its class and import path, and a single generic renderer produces the constructor call. Only blocks with a different code shape (dataset loading with a train/test split, a PyTorch model definition, and the training step) use hand-written templates, chosen by a dispatcher. Adding a standard block therefore needs one table entry. The steps are assembled into a scikit-learn `Pipeline` and shown in a read-only code panel.

D. Rules-Based Preprocessing Suggestions

A dataset profiler computes column types, missing-value percentage, cardinality, skewness, outlier rates, date-like patterns, and class imbalance. Deterministic rules map each issue to a pre-built fix block. Each suggestion appears as a plain-language card with Accept, Skip, and "What does this mean?" options; nothing is applied silently. Rules were chosen over a learned detector for determinism, speed, and exact explainability.

E. Bounded Use of LLMs

Initial draft with pre-filled parameters. The user answers plain-language questions (what to predict, which column is the target, category or number). The backend combines the answers with the dataset profile in a fixed internal prompt asking the LLM for blocks *and* parameter values. The backend validates each value against the block's schema and falls back to library defaults when a value is invalid. Users are not asked to choose parameters; each block explains in one sentence why its values were set, and sliders remain optional. Alternative blocks in the palette are chosen by rules (valid substitutions for that step).

Custom code for unmatched cases. When the profiler detects an issue that no pre-built block covers, the issue is turned into a prompt automatically and the LLM drafts a small function placed in a "Custom Code" block. The block is labeled AI-generated, carries a plain-language explanation, and requires Accept, Edit, or Reject before it runs.

F. Live Metrics and Playground

On each change or run, the backend trains the model and streams accuracy, loss, and the confusion matrix to the interface. A try-it playground exposes the trained model through an inference endpoint for live predictions.

G. Interfaces

REST endpoints are grouped into dataset, pipeline, model, and block resources: upload and profile datasets; create, update, run, and fetch metrics and code for pipelines; predict with and export models; and list the block catalog and its documentation.

IV. IMPLEMENTATION

The frontend uses React, Tailwind CSS, and a flow-diagram library. The backend is Python with FastAPI. Classic ML uses scikit-learn, and PyTorch deep learning is a stretch goal. Datasets can be imported through the Kaggle API, and metadata is stored in SQLite or PostgreSQL. LLM calls use a hosted API, chosen for low and predictable latency and reliable structured output compared with self-hosting small open models.

V. EVALUATION METHODOLOGY

A. Research Questions

- **RQ1.** Do novices using ModelForge with transparency features (visible code, block explanations) show greater

gain in conceptual understanding than novices using a version without them?
- **RQ2.** How does transparency affect trust, confidence, and mental workload?
- **RQ3.** How usable is ModelForge?
- **RQ4 (technical).** Do LLM-chosen parameters perform at least as well as library defaults, and how accurately does the profiler detect seeded data issues?

B. Participants and Design

The study targets 20 to 30 undergraduate students with no ML coursework. A between-subjects design randomly assigns participants to (A) the full interface or (B) the same interface with the code panel and explanations hidden. Participation is voluntary and based on informed consent, with responses kept anonymous.

C. Tasks and Measures

Each participant completes the same two tasks: build a classifier for a small tabular dataset, then improve its accuracy. Measures:

- **Learning:** a pre-test and post-test of about ten multiple-choice items on preprocessing, model choice, and metrics; the gain is the post-test score minus the pre-test score.
- **Trust and confidence:** a short Likert questionnaire.
- **Usability:** the System Usability Scale (SUS).
- **Mental workload:** NASA-TLX, which allows comparison with workload findings reported for LLM-assisted visual programming [3].

D. Technical Evaluation (RQ4)

On several public tabular datasets, model performance with LLM-chosen parameters is compared against library defaults. Datasets are also seeded with known issues (missing values, outliers, skew, imbalance), and the profiler's detection precision and recall are reported.

E. Analysis Plan

Learning gains are compared across conditions with an independent-samples t-test, or a Mann–Whitney U test if normality fails, reporting effect sizes and confidence intervals; the pre-test score may enter as a covariate. Questionnaire data are analyzed in the same way, and open-ended comments are coded thematically.

VI. RESULTS

The study described above has not yet been conducted, so no empirical results are reported in this version. Pre-test, post-test, learning gain, trust, SUS, and NASA-TLX scores for each condition, with test statistics and effect sizes, will be reported once data collection is complete.

VII. DISCUSSION AND LIMITATIONS

The discussion of findings will follow data collection. Expected limitations are a small, single-institution sample; learning measured only shortly after use; a rules-based profiler limited to predefined issue patterns; and LLM-generated code for novel cases, which stays non-deterministic even with mandatory review.

VIII. CONCLUSIONS

This paper presented ModelForge, a no-code ML platform that keeps one pipeline specification as the source of truth for training, metrics, and readable code, and restricts AI to bounded, reviewable roles. A controlled study was proposed to test whether this transparency improves novices' conceptual learning. Future work includes deep learning and GenAI pipelines, Hugging Face integration, an offline mode for privacy-sensitive users, collaborative projects, and a larger, longer-term study.

REFERENCES

[1] J. Drozdal et al., "Trust in AutoML: Exploring information needs for establishing trust in automated machine learning systems," Proc. 25th Int. Conf. Intelligent User Interfaces (IUI), 2020.

[2] R. Du et al., "Rapsai: Accelerating machine learning prototyping of multimedia applications through visual programming," Proc. CHI Conf. Human Factors in Computing Systems, 2023.

[3] Z. Zhou et al., "InstructPipe: Generating Visual Blocks pipelines with human instructions and LLMs," Proc. CHI Conf. Human Factors in Computing Systems, 2025, arXiv:2312.09672.

[4] H. Ali, J. Galasso-Carbonnel, and H. Sahraoui, "From intent to AI pipelines: A controlled agentic framework for non-AI expert scientists," arXiv:2605.18764, 2026.

[5] B. Balis, M. Orzechowski, P. Kica, M. Dygas, and M. Kuszewski, "From research question to scientific workflow: Leveraging agentic AI for science automation," arXiv:2604.21910, 2026.

[6] N. Rao, J. Tsay, K. Kate, V. J. Hellendoorn, and M. Hirzel, "AI for low-code for AI," arXiv:2305.20015, 2023.

[7] M. Truss and M. Schmitt, "Human-centered AI product prototyping with no-code AutoML: Conceptual framework, potentials and limitations," arXiv:2402.07933, 2024.

[8] M. Carney et al., "Teachable Machine: Approachable web-based tool for exploring machine learning classification," Ext. Abstracts CHI Conf. Human Factors in Computing Systems, 2020.

[9] M. Strobel, "Understanding the effects of GenAI as no-code alternative for teaching machine learning workflows," AAAI Conf. Artificial Intelligence (EAAI), 2026.

[10] R. Leenings et al., "The PHOTON Wizard – Towards educational machine learning code generators," arXiv:2002.05432, 2020.

[11] N. Abarca, A. Carvallo, C. L. Moncada, and F. Bravo-Marquez, "Explaining AI without code: A user study on explainable AI," arXiv:2602.11159, 2025.